SKILLPROOF

Blockchain-Based Credential Verification DApp

Project Type Decentralized Application (DApp)	**Blockchain** Ethereum Sepolia
Frontend HTML, CSS, JavaScript, Ethers.js	**Wallet** MetaMask

A practical blockchain solution for issuing, owning, sharing, and independently verifying digital skill and achievement credentials.

Deployed Smart Contract

```
0x7814437a0Dda2F03cC6a6544AAf95eC0bC7a698f
```

Network: Ethereum Sepolia

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Proposed Solution
- 6. System Workflow
- 7. System Architecture
- 8. Smart Contract Design
- 9. Frontend Modules
- 10. Credential Lifecycle
- 11. Verification Logic
- 12. Technology Stack
- 13. Testing and Validation
- 14. Advantages
- 15. Limitations
- 16. Future Scope
- 17. Conclusion

1. Abstract

- SkillProof is a blockchain-based credential verification DApp designed to make digital skill and achievement credentials easier to issue, own, share, and verify. The system stores credential records on the Ethereum Sepolia test network through a Solidity smart contract. Authorized issuers can create credentials for a wallet address, while credential holders can retrieve credentials associated with their wallet. A public verification interface allows anyone to check a credential using its unique credential ID without requiring a wallet connection.
- The project combines a static web frontend, Ethers.js, MetaMask, and an on-chain smart contract. The implemented workflow supports credential issuance, credential retrieval, public verification, and credential revocation.

2. Introduction

- Digital certificates are commonly used to represent academic achievements, training, internships, workshops, and technical skills. A conventional certificate often depends on the issuing organization or a centralized verification process.
- SkillProof approaches this problem by placing the core credential record on a blockchain. The credential is represented by an on-chain record containing its identifier, holder address, title, issuer name, issue timestamp, and status.

3. Problem Statement

- Traditional digital certificates can be difficult for a third party to independently verify.
- A recipient may not have a simple way to prove that a credential was genuinely issued to them.
- Recruiters, institutions, or other verifiers may need to contact the issuing organization for confirmation.
- A certificate can be copied or modified without changing the appearance of the document.
- There is a need for a simple verification mechanism based on a unique credential identifier.

4. Objectives

- Create a blockchain-backed system for issuing digital skill and achievement credentials.
- Give every credential a unique credential ID.
- Associate credentials with the recipient's blockchain wallet address.
- Allow authorized issuers to issue and revoke credentials.
- Allow recipients to view credentials associated with their wallet.
- Allow anyone to verify a credential publicly using its credential ID.
- Provide a clean and usable interface for issuers, recipients, and verifiers.

5. Proposed Solution

- SkillProof separates the application into three practical user roles: issuer, credential holder, and verifier. The issuer uses the Issuer Portal to create a credential. The holder uses My Credentials to retrieve credentials associated with their wallet. A verifier uses the public Verify page to inspect a credential by ID.

6. System Workflow

Credential issuance flow

- The issuer connects MetaMask to Ethereum Sepolia.
- The issuer enters the recipient wallet address, achievement/skill title, and issuer name.
- The frontend calls the deployed SkillProof smart contract.
- The smart contract creates a new credential record and assigns a unique credential ID.
- The transaction is confirmed on Sepolia.
- The credential becomes retrievable for the recipient wallet and verifiable through the public verification interface.

Credential holder flow

- The recipient connects the wallet that received the credential.
- The My Credentials page calls getMyCredentials().
- The returned credential IDs are read from the blockchain.
- Credential details and status are displayed as credential cards.
- The dashboard summarizes total, verified, and revoked credentials.

Public verification flow

- A verifier enters a credential ID such as SP-000002.
- The application reads the credential record from the blockchain.
- The page displays the credential title, holder, issuer, issue date, and validity state.
- A wallet is not required for public verification.

7. System Architecture

Layer	Component	Responsibility
Presentation	HTML / CSS / JavaScript	User interface, navigation, forms, cards and states
Blockchain Interface	Ethers.js	Calls smart-contract functions and reads blockchain data
Wallet	MetaMask	Connects issuer/holder wallet and signs issuer transactions
Smart Contract	SkillProof.sol	Stores, issues, verifies and revokes credentials
Blockchain	Ethereum Sepolia	Persistent on-chain credential records
Public Read	Sepolia RPC	Allows wallet-free credential verification

8. Smart Contract Design

The deployed SkillProof contract uses Solidity ^0.8.20. The contract maintains credential records and issuer authorization. The deployer is the contract owner and is initially authorized as an issuer.

Credential data model

Each credential contains: credentialId, holder, title, issuerName, issuedAt, revoked, and exists. This structure identifies the credential, associates it with a holder wallet, stores descriptive information, records issue time, and determines revocation state.

Core contract functions

```
authorizeIssuer(address issuer, bool status)

issueCredential(address holder, string title, string issuerName)

getMyCredentials()

getHolderCredentials(address)

getCredential(uint256)

credentialExists(uint256)

verifyCredential(uint256)

revokeCredential(uint256)
```

The contract also exposes CredentialIssued, CredentialRevoked, and IssuerAuthorized events for important state changes.

9. Frontend Modules

Home Page — Project introduction, branding, navigation, and entry points.

Issuer Portal / Admin — Authorized issuers connect a wallet, enter recipient and credential data, submit the transaction, and receive the resulting credential ID.

My Credentials — Displays credentials belonging to the connected recipient wallet, including total, verified, revoked counts and credential details.

Verify — Public verification by credential ID without requiring a wallet connection.

10. Credential Lifecycle

- **1. Issue** — An authorized issuer submits a credential transaction.
- **2. Record** — The smart contract stores the credential on Ethereum Sepolia.
- **3. Identify** — The credential receives a unique numeric ID, displayed as an SP-prefixed identifier.
- **4. Retrieve** — The holder retrieves credentials using the wallet-associated credential list.
- **5. Verify** — Anyone can inspect the credential using its ID.
- **6. Revoke** — An authorized issuer can revoke the credential; verification then reports it as no longer valid.

11. Verification Logic

The verification interface distinguishes three states: a valid credential exists on-chain and has not been revoked; a revoked credential exists but is no longer valid; and a missing credential ID has no corresponding record and is reported as not found.

Example

```
SP-000002 → existing credential → display credential information and current validity state.

SP-000007 → if no such credential exists → display credential not found.
```

12. Technology Stack

Technology	Use in Project
Solidity	Smart-contract development
Ethereum Sepolia	Blockchain deployment and testing
Ethers.js v6	Frontend-to-blockchain interaction
MetaMask	Wallet connection and transaction signing
HTML5	Page structure
CSS3	Visual design and responsive UI
JavaScript	Application logic and blockchain calls
Vercel	Static frontend deployment target

13. Testing and Validation

The implemented DApp was tested through the deployed contract and frontend workflow on Ethereum Sepolia.

Test Area	Expected / Observed Behavior
Issuer authorization	Authorized issuer can access issuance flow.
Credential issuance	Transaction is submitted through MetaMask and a credential ID is generated.
Credential retrieval	Recipient wallet retrieves its associated credential IDs.
Credential statistics	My Credentials displays total, verified and revoked counts.

Public verification	Existing credential IDs return credential details and validity.
Invalid credential	Non-existing ID is reported as not found.
Revocation	Revoked credentials are displayed as revoked and are no longer treated as valid.

14. Advantages

- Public verification reduces dependence on manual issuer confirmation.
- The credential record is stored on-chain rather than only in a local document.
- A unique credential ID makes individual credentials easy to reference.
- Wallet association gives the recipient a direct way to retrieve credentials.
- Revocation provides a mechanism for invalidating a previously issued credential.

15. Limitations

- The current implementation is deployed and tested on the Ethereum Sepolia test network.
- The system stores the core credential record on-chain; richer off-chain metadata would require additional design.
- The credential model is intentionally lightweight and does not implement a standardized credential format.
- Wallet ownership is used as the holder identity, so users must retain access to the relevant wallet.

16. Future Scope

- Deploy to a production blockchain network after appropriate security and cost evaluation.
- Add QR codes that open a credential verification page.
- Support richer credential metadata and optional document/evidence references.
- Improve issuer administration and organization-level identity management.
- Introduce more advanced credential sharing and presentation features.

17. Conclusion

SkillProof demonstrates how blockchain can be used to create a practical, publicly verifiable credential system. The completed prototype connects a web interface with a Solidity smart contract on Ethereum Sepolia, enabling authorized issuers to issue and revoke credentials, recipients to view credentials associated with their wallets, and third parties to verify credentials independently using a unique credential ID.

The project provides a complete DApp workflow rather than a static certificate mockup: credential lifecycle is backed by blockchain state, wallet interaction is integrated through MetaMask, and public verification is available through a dedicated interface.

Project Reference

Smart Contract: 0x7814437a0Dda2F03cC6a6544AAf95eC0bC7a698f

Network: Ethereum Sepolia

Project: SkillProof — Blockchain-Based Credential Verification DApp